

Monitoring Plan and Incident Response Playbook - Northstar Assist

Tarie Nosworthy

18 September 2026

Contents

Northstar Assist — Internal Employee RAG Assistant	1
Part 1 — Monitoring Plan	2
1.1 Log sources	2
1.2 The real log structure	2
1.3 AI-specific signals and alert conditions	3
1.4 Signal explicitly NOT implemented	4
1.5 Attribution gap — must be fixed before production	4
1.6 Useful CloudWatch Logs Insights queries	4
Part 2 — Incident Response Playbook	5
Suspected prompt injection incident	5
Step 0 — Preconditions	5
Step 1 — Confirm the alert is real (target: 5 minutes)	5
Step 2 — Determine whether anything was disclosed (target: 15 minutes)	6
Step 3 — Contain (decision point)	6
Step 4 — Check corpus integrity (target: 20 minutes)	7
Step 5 — Verify the controls still work (target: 10 minutes)	7
Step 6 — Escalate, if any of these are true	7
Step 7 — Recover and record	8
Appendix — Known control gaps an analyst should expect	8

Northstar Assist — Internal Employee RAG Assistant

Assessment date: 18 September 2026 **Author:** Tarie Nosworthy **Environment:** AWS account 118924230273, us-east-1

Every filter pattern below was written against log records **observed in this account**, not from documentation. Where a desirable signal turned out not to be derivable from the available logs, it is recorded as a gap rather than shipped as a pattern that looks plausible and silently counts nothing.

Part 1 — Monitoring Plan

1.1 Log sources

Source	Log group / mechanism	Contents	Notes
Bedrock model invocation logs	/northstar-assist/ model-invocations	Full prompts, completions, stopReason, guardrail assessments	Primary source. Off by default, enabled by scripts/60_monitoring.py --enable-logging. Delivery role is write-only to this one group with confused-deputy conditions.
Harness runtime logs	/aws/bedrock-agentcore/ runtimes/harness_NorthstarAssist- <id>-DEFAULT	Agent loop execution, tool invocations, retrieval detail	The only source that sees chunk counts — see §1.4.
Knowledge base application logs	/aws/vendedlogs/ bedrock/knowledge-base/ APPLICATION_LOGS/<kb- id>	Ingestion, sync results, per-document status	Critical for corpus-integrity monitoring (threat T-01).
CloudTrail	Management events	UpdateHarness, UpdateGuardrail, PutRolePolicy, CreatePolicyVersion, S3 writes to the corpus bucket	Change control. Not configured by this assessment; required for production .
Client-side instrumentation	Application layer	retrieved_chunks per turn, session correlation, authenticated user	scripts/harness_client.py demonstrates this; the reference Streamlit app does not implement it.

A limitation to state plainly. The harness event stream carries **no guardrail trace event**. Intervention has to be inferred from `$.output.outputBodyJson.stopReason` and the guardrail’s blocked-message text. That is why the two blocked messages are worded distinctly — it is the only way to distinguish an input-side from an output-side intervention from the response alone.

CloudWatch GenAI Observability traces require Transaction Search, which was not available in the assessment environment. In production it would supersede several of the inferences below and is recommended.

1.2 The real log structure

Four corrections came out of sampling actual records, each of which would have produced a silently-broken filter:

1. The guardrail trace lives at `$.output.outputBodyJson.trace.guardrail` — **not** under an `amazon-bedrock-trace` key.
2. Assessments are keyed by **guardrail ID** (`inputAssessment.6yxbopugn44t...`), not by a wildcard-friendly structure.
3. CloudWatch Logs permits **one wildcard per JSON selector**. A pattern using `inputAssessment.*.contentPolicy.filters[*]` used two and was **rejected outright** by `PutMetricFilter` — a visible failure, fortunately.
4. `$.output.outputBodyJson.stopReason` carries `guardrail_intervened`, which is a simpler and more reliable intervention signal than any assessment field.

Verified fields available per record: `operation` (`ConverseStream`), `identity.arn` (the harness execution role), `output.outputTokenCount`, `input.inputTokenCount`, `output.outputBodyJson.usage.{inputTokens, outputTokens,`

totalTokens}, output.outputBodyJson.stopReason, output.outputBodyJson.metrics.latencyMs, and the guardrail trace.

1.3 AI-specific signals and alert conditions

Namespace NorthstarAssist/Security. All five are deployed and were confirmed in OK state.

Signal 1 — GuardrailInterventions

```
{ $.output.outputBodyJson.stopReason = "guardrail_intervened" }
```

Alert: Sum > 10 per 1 hour

Rationale. A handful of interventions per hour is normal for an internal assistant — people phrase things carelessly. Ten in one hour from a population this size is either a user probing deliberately or a broken prompt template in a calling application. Measured baseline for calibration: the hardened test suite produced **55 interventions across ~78 invocations**, but that was adversarial by construction. Normal traffic should sit near zero.

Verified: stopReason observed in live records. No wildcards.

Signal 2 — PromptAttackInterventions (*headline alert*)

```
{ $.output.outputBodyJson.trace.guardrail.inputAssessment.<guardrail-id>.contentPolicy.filters[*].type = "PROMPT_ATTACK" }
```

Alert condition — the concrete one required by the rubric:

Alarm when PromptAttackInterventions Sum > 5 in any 1-hour period. Namespace NorthstarAssist/Security, statistic Sum, period 3600s, 1 evaluation period, TreatMissingData: notBreaching, ComparisonOperator: GreaterThanThreshold. Action: notify the security on-call SNS topic.

Why 5 per hour. One or two prompt-attack detections are consistent with curiosity or an employee pasting content that happens to read as imperative. Five in one hour is not accidental — it is iteration, which is what probing looks like. Set lower and normal curiosity pages the on-call; set higher and a methodical attacker works unobserved for hours. The threshold assumes a few hundred employees; it should be re-derived from two weeks of production traffic before being treated as tuned.

Verified: path structure observed; guardrail ID templated in to stay within the one-wildcard limit.

Signal 3 — GroundingBlocks

```
{ $.output.outputBodyJson.trace.guardrail.outputAssessments[0].<guardrail-id>.contextualGroundingPolicy.filters[*].action = "BLOCKED" }
```

Alert: Sum > 8 per 1 hour

Rationale — this is the corpus-integrity signal. Contextual grounding was the only control that caught corpus poisoning in testing, blocking 2 of 3 attempts (test DP-01). A rising grounding-block rate is what a poisoned document looks like *from the outside*: a document contradicting the rest of the corpus depresses groundedness scores for every query it matches. It also rises when the corpus has drifted from what users actually ask about, which is worth knowing for different reasons.

Operational note. If the recommended threshold change (0.75 → 0.60) is applied, this alarm's baseline drops and the threshold must be re-derived. The change trades preventive strength for usability; this alarm is the compensating detection and must not be left un-recalibrated.

Signal 4 — HighTokenResponses

```
{ $.output.outputTokenCount > 3000 }
```

Alert: Sum > 5 per 1 hour

Rationale. Normal policy answers run a few hundred output tokens; the measured mean across 74 hardened runs was **4,159 total tokens** with a **maximum of 8,590**. Repeated multi-thousand-token *output* is either a cost attack or bulk extraction of corpus content.

This signal has a specific provenance. In test PI-06 the token cap was **the only thing that stopped** a forged-context attack midway through dumping the employee directory (stopReason: max_tokens, 3,400 characters emitted). A token cap is not a security control — but the *signal* that one was hit is a high-value indicator that something tried to extract in bulk.

Signal 5 — ToolCallFailures

```
{ $.output.outputBodyJson.stopReason = "malformed_tool_use" }
```

Alert: Sum > 5 per 1 hour

Rationale. A reliability signal with a security consequence. Observed live in test FP-05 run 2: Nova 2 Lite emitted a tool call the runtime could not parse, producing an empty response. A failed tool call yields an answer with no retrieval behind it — **indistinguishable to the user from a grounded one**. A rising rate means users are receiving ungrounded answers while the system reports success.

1.4 Signal explicitly NOT implemented

ZeroChunkRetrieval — responses produced with no knowledge base retrieval.

This is a desirable signal and a natural one to ask for. It is **not derivable from model invocation logs**, and the honest thing is to say so rather than ship a pattern that appears to work:

- Retrieval happens in the **AgentCore Gateway**, not in the model call. A ConverseStream record shows the tool *configuration* but not how many chunks came back.
- A single user turn produces **several** invocation records (decide-to-call, then answer-with-results), so per-record reasoning about retrieval is ambiguous.

An initial pattern keyed on toolConfig NOT EXISTS was accepted by PutMetricFilter without error and would have counted approximately nothing — the worst failure mode, because the alarm exists and stays green.

Where it must come from instead: the harness runtime log group, or client-side instrumentation. scripts/harness_client.py records retrieved_chunks per turn directly and is the reference implementation. **Recommendation:** implement this client-side before production; it is the detection for threat E-02 and for the malformed_tool_use blind spot.

1.5 Attribution gap — must be fixed before production

Invocation logs identify the caller as the **harness execution role**:

```
arn:aws:sts::118924230273:assumed-role/AmazonBedrockAgentCoreHarnessDefaultServiceRole-fpraf/BedrockAgentCore-<uuid>
```

Not the employee. Every user's activity attributes to one shared principal, so "*which employee extracted the directory*" is **unanswerable from the logs**. InvokeHarness accepts runtimeUserId and actorId; neither is populated by the reference integration.

Without this, the containment steps in Part 2 cannot be scoped to a user. This is threat R-01 and it gates several playbook steps.

1.6 Useful CloudWatch Logs Insights queries

Interventions by hour:

```
fields @timestamp, @message
| filter output.outputBodyJson.stopReason = "guardrail_intervened"
| stats count() as interventions by bin(1h)
| sort @timestamp desc
```

Highest-token responses (extraction candidates):

```
fields @timestamp, output.outputTokenCount, output.outputBodyJson.stopReason
| filter output.outputTokenCount > 2000
| sort output.outputTokenCount desc
| limit 50
```

Requests that hit the token cap:

```
fields @timestamp, output.outputTokenCount
| filter output.outputBodyJson.stopReason = "max_tokens"
| stats count() by bin(1h)
```

Token consumption per hour (cost and amplification):

```
fields @timestamp, output.outputBodyJson.usage.totalTokens as total
| stats sum(total) as tokens, count() as calls, avg(total) as mean by bin(1h)
```

Part 2 — Incident Response Playbook

Suspected prompt injection incident

Audience: an on-call analyst with no prior AI incident experience. Every step gives an exact console path or command. Do not skip the containment decision at step 3 — it is the only irreversible one.

Step 0 — Preconditions

You need: AWS console access to account 118924230273 (us-east-1), permission to read CloudWatch Logs, and permission to run UpdateHarness and UpdateGuardrail. If you do not have the last of those, escalate immediately — you cannot contain without it.

Reference values from the deployment:

Item	Value
Harness	NorthstarAssist
Log group	/northstar-assist/model-invocations
Guardrail	northstar-assist-guardrail
Corpus bucket	northstar-assist-kb-<suffix>
Metric namespace	NorthstarAssist/Security

Step 1 — Confirm the alert is real (target: 5 minutes)

Console → **CloudWatch** → **Alarms** → click the alarming alarm → **View data in metrics**. Note the exact hour that breached.

Then read the records behind it. Console → **CloudWatch** → **Logs Insights** → select /northstar-assist/model-invocations → set the time range to the breaching hour → run:

```
fields @timestamp, input.inputBodyJson.messages.0.content.0.text as prompt,
      output.outputBodyJson.stopReason as stop
| filter output.outputBodyJson.stopReason = "guardrail_intervened"
| sort @timestamp desc
| limit 100
```

Decide: are these genuine attack attempts, or one malformed calling application repeating the same request? A single repeated identical prompt is usually a bug. Varied, escalating rephrasings are probing.

If it is a bug: raise a ticket against the calling application, silence the alarm with an explicit expiry, and stop here. Do not proceed to containment.

Step 2 — Determine whether anything was disclosed (target: 15 minutes)

Interventions mean the controls *fired*. The question is whether anything got through **before** they did.

Find high-volume responses in the window:

```
fields @timestamp, output.outputTokenCount, output.outputBodyJson.stopReason
| filter output.outputTokenCount > 1500
| sort output.outputTokenCount desc
| limit 50
```

Then read the full text of the largest:

```
fields @timestamp, @message
| filter output.outputTokenCount > 1500
| sort @timestamp desc
| limit 5
```

Look specifically for — these are the corpus’s actual sensitive shapes:

- Tabular employee data, or any EMP### identifier
- CUST### or OPP-YYYY-NNN alongside monetary figures
- Any string matching ns_live_... or ns_test_... (**a credential — escalate immediately, see step 6**)
- stopReason: max_tokens on a large response — in testing this was the signature of a **partially completed bulk extraction**

Record the runtimeSessionId of anything suspicious. Note the limitation from §1.5: the logs will not tell you *which employee* this was. If runtimeUserId is not populated, escalate to the application owner to correlate the session against their access logs — you cannot do it from here.

Step 3 — Contain (decision point)

Choose the least disruptive option that stops the harm.

Option A — tighten the guardrail (preferred; preserves service).

Console → **Amazon Bedrock** → **Guardrails** → northstar-assist-guardrail → **Edit**. Raise MISCONDUCT to HIGH, add a denied topic matching the observed attack pattern, and/or set GROUNDING to 0.85. Then **Create version** and note the new version number.

Point the harness at it: **AgentCore** → **Harness** → NorthstarAssist → **Edit** → Parameters → guardrailConfig → change guardrailVersion to the new version → **Save**.

⚠ **Verify the tool survived.** GetHarness returns fields UpdateHarness rejects, and a console save can silently drop configuration. After saving, confirm the **Gateway** tool is still attached — a harness that answers without retrieval looks healthy and is ungrounded. Or run:

```
.\.venv\Scripts\python.exe submission\scripts\30_guardrail.py --status
```

Option B — disconnect retrieval (service degraded, corpus protected).

AgentCore → **Harness** → NorthstarAssist → **Edit** → **Tools** → toggle **Gateway** off → **Save**. The agent keeps answering but can no longer reach any Northstar document. Use this when the concern is *data exposure* and you need the corpus unreachable within minutes.

Option C — full stop (service down).

Remove the harness's ability to invoke the model. Console → **IAM** → Roles → AmazonBedrockAgentCoreHarnessDefaultServiceRole-fpraf → the customer-managed ...HarnessExecutionPolicy... → edit the BedrockModelInvocation statement Effect to Deny, or detach the policy. Every request then fails closed.

Prefer this over deleting the harness: deletion destroys the configuration you need for forensics, and the harness ID appears throughout the logs you are about to read.

⚠ **The containment path depends on credentials.** During this assessment a credential revocation left live agent infrastructure running and unreachable — the operator could neither inspect nor decommission it (threat D-02). If your credentials fail here, use the break-glass role. If no break-glass role exists, that is the finding to raise after the incident.

Step 4 — Check corpus integrity (target: 20 minutes)

Do this even if the incident looks like direct injection. A successful injection and a poisoned document produce similar symptoms, and poisoning is the higher-impact root cause — in testing it succeeded in 1 of 3 attempts and the agent *cited the poisoned document as its source*.

Who wrote to the bucket recently? Console → **CloudTrail** → **Event history** → filter Event source s3.amazonaws.com, Event name PutObject, resource = the corpus bucket. Any write not from the approved publishing pipeline is a candidate.

What did the last ingestion do? Console → **AgentCore** → **Knowledge Bases (KB)** → your KB → the data source → **Sync history**. Look for documents indexed or modified that you cannot account for.

Scan the corpus for injection-shaped and credential content:

```
.\.venv\Scripts\python.exe submission\scripts\scan_corpus.py
```

This reports per-document classification and flags credential material, direct-identifier PII, and **imperative instruction text**. The supplied corpus scored **zero** imperative_instruction matches — that clean baseline is the comparison. Any hit is a candidate poisoned document.

If a poisoned document is found: delete it from S3, re-sync the data source (the S3 delete alone does not remove it from the **index**), then re-run the scan to confirm.

Step 5 — Verify the controls still work (target: 10 minutes)

Do not declare the incident closed on the basis that alarms stopped firing. Alarms also stop firing when a control breaks.

```
.\.venv\Scripts\python.exe submission\scripts\40_run_tests.py --mode hardened --runs 1
```

29 tests, ~30 invocations, a few minutes. Compare against docs/test-results-hardened.md. Expect attack block rate ≈ 0.939 and the five FP-* tests to behave as they did before.

Two specific checks:

- **Over-blocking:** if the FP-* tests now fail, step 3's tightening made the assistant unusable. That is its own incident.
- **Retrieval:** if attack tests pass but FP-01 returns no chunks, retrieval is broken and every answer is now ungrounded.

Attribute any specific block precisely rather than guessing:

```
.\.venv\Scripts\python.exe submission\scripts\32_guardrail_probe.py --suite
```

This calls ApplyGuardrail directly and returns the policy name and confidence for each detection. Note that **contextual grounding cannot be evaluated by text-only probing** — it needs grounding_source and query as separate qualified blocks, or it silently returns NONE.

Step 6 — Escalate, if any of these are true

- **A credential was disclosed** (ns_live_... in any response). Page the platform team to **rotate the token immediately**. Note that api_authentication_guide.html contains a live-format token by default — confirm whether the corpus was ever remediated.

- **Employee PII left the system in bulk** — engage HR and privacy; there may be notification obligations.
- **Customer commercial terms were disclosed** — engage the account team and Legal.
- **A poisoned document was found** — treat as an integrity incident, not just an AI one. Determine who had bucket write access and how.
- **Containment failed** because credentials were revoked or insufficient — escalate to the account owner; this is an availability incident in its own right.

Step 7 — Recover and record

1. Restore service: re-enable the Gateway tool (Option B) or restore model invocation permissions (Option C).
 2. Re-run step 5 and confirm attack block rate and the FP-* tests.
 3. Confirm the alarms are in OK and un-silenced.
 4. Record: the alert that fired, prompts observed, whether anything was disclosed, containment option used, root cause, and — explicitly — **whether the incident could be attributed to an individual user**. If not, that is a finding about §1.5, not a dead end.
 5. If a new attack pattern got through, **add it to submission/tests/prompts.yaml** with an expectation. The suite is the regression test; an attack that is not in it will not be checked for again.
-

Appendix — Known control gaps an analyst should expect

An analyst working this playbook will meet these. They are documented so the analyst does not waste time looking for a control that does not exist.

Gap	Consequence for IR
Retrieved content is not screened by the input guardrail	A prompt-attack alarm will never fire for indirect injection. Absence of alarms is not absence of injection. Use step 4.
Forged assistant turns are undetected	Conversation history is caller-supplied (Memory disabled). The logged “assistant” turns may be attacker-authored. Do not treat logged history as a record of what the agent said.
No guardrail trace event in the response stream	Intervention is inferred from stopReason and blocked-message text.
No end-user attribution	Sessions cannot be tied to employees without application-side correlation.
The agent fabricates action completion	It claimed to have deleted a document it has no tool to delete. Never accept the agent’s report of its own actions as evidence — verify against S3 and CloudTrail.
ZeroChunkRetrieval is not monitored	Ungrounded answers are not currently detectable from invocation logs.
No rate limiting	Nothing throttles an attacker mid-incident. Containment is all-or-nothing.